UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. Timer management.
8. Congestion control.

The Internet Transport Protocol: UDP

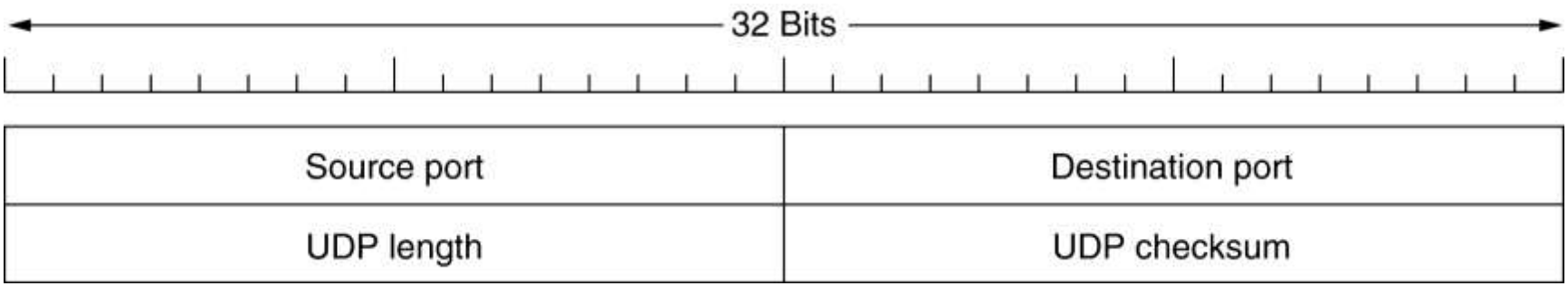
- Introduction to **UDP** (User Datagram Protocol).
- Segment header

Two applications of UDP:

1. Remote Procedure Call (**RPC**).
2. The Real-Time Transport Protocol (**RTP**).

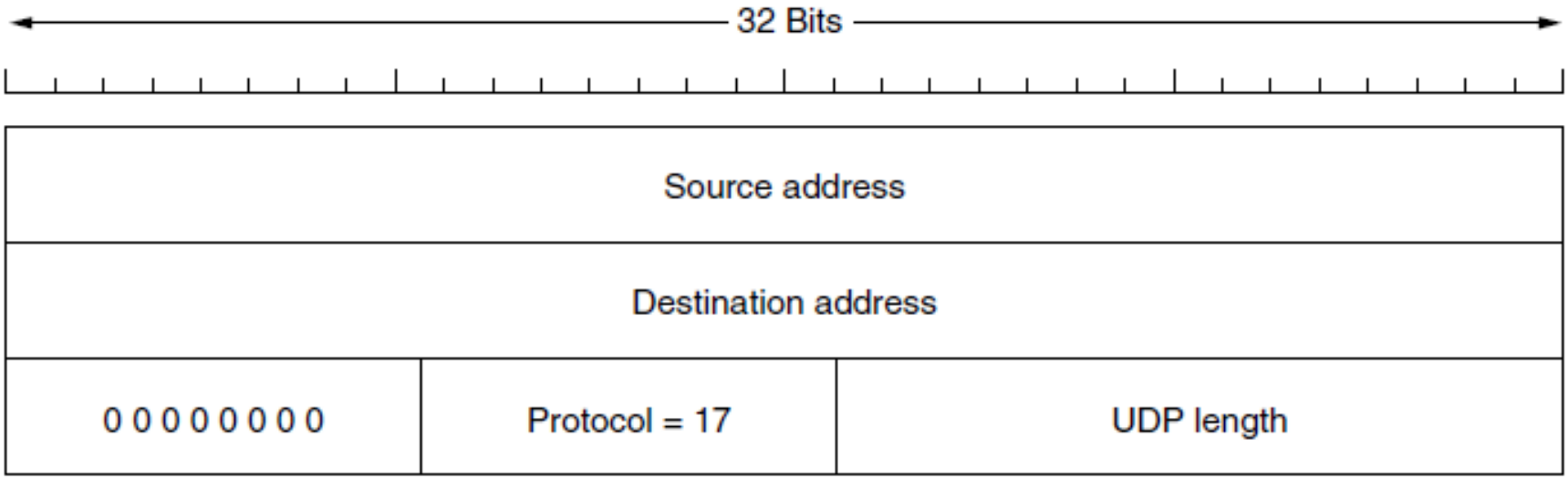
Introduction to UDP

- The Internet has two main protocols in the Transport Layer, a Connectionless Protocol and Connection-oriented Protocol.
- **UDP** is a transport layer protocol that typically runs in the operating system and protocols that use UDP typically run in user space.



The UDP Header.

- The packet produced by the UDP is called a **user datagram.**
  - **Source port address:** The source port address is the address of the application program that has created the message.
  - **Destination port address:** The destination port address is the address of the application program that will receive the message.
  - **Total length:** The total length field defines the total length of the user datagram in bytes.
  - **Check sum:** The check sum is a 16-bit field used in error detection.



The IPv4 pseudoheader included in the UDP checksum

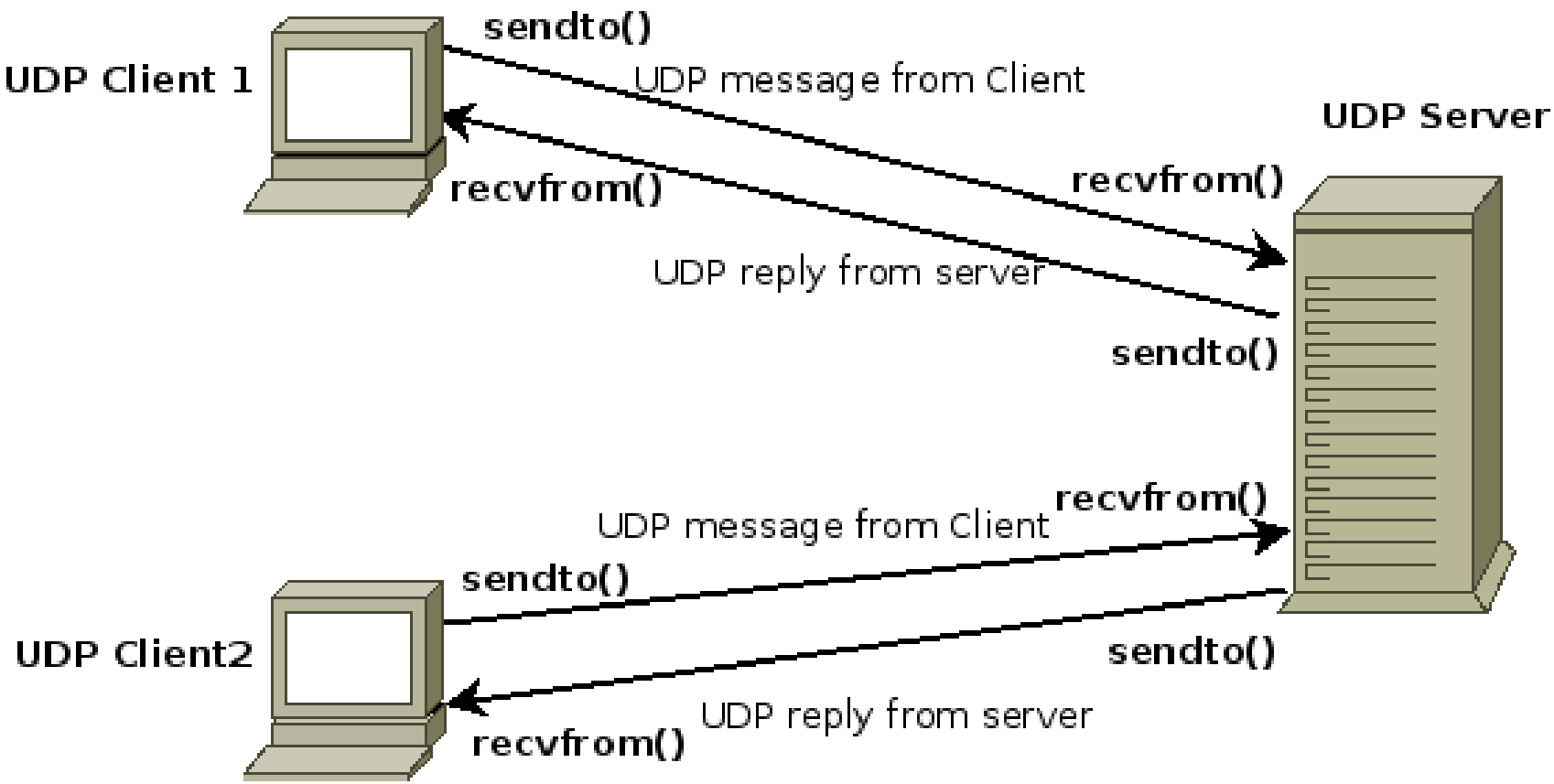
- Including the pseudoheader in the UDP checksum computation helps detect misdelivered packets.
- UDP can discover that an error has occurred; ICMP can then inform the sender that a user datagram has been damaged and discarded.

- It is probably worth mentioning explicitly some of the things that **UDP does not do.**
  - **It does not do Flow Control, Error Control, or Retransmission upon receipt of a Bad Segment.**
  - **All of that is up to the User Processes.**
- What it does do is provide an interface to the IP protocol with the added feature of demultiplexing multiple processes using the ports.
- One area where UDP is especially useful is in client-server situations. Often, the client sends a short request to the server and expects a short reply back.

Applications of UDP

- **Client- Server Situations**: The client sends a short request to the server and expects a short reply back. If either the request or the reply is lost, the client can just time out and try again.
- **DNS (Domain Name System):** for example, www.cs.berkeley.edu, can send a UDP packet containing the host name to a DNS server. The server replies with a UDP packet containing the host's IP address. No setup is needed in advance and no release is needed afterward. Just two messages go over the network.

UDP - Client Server Application

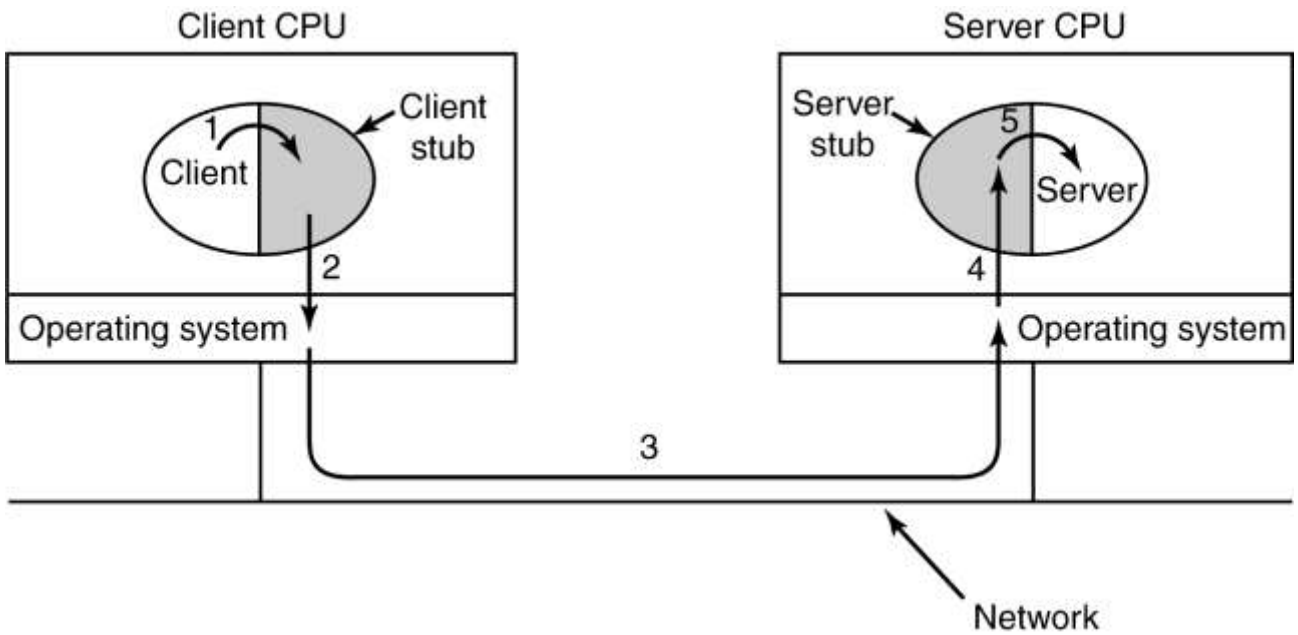


Transmission Control Protocol (TCP)	User Datagram Protocol (UDP)
TCP supports **host-to-host** communication I,e. Connection oriented protocol	UDP enables **process-to-process** communication, I,e. Connectionless protocol
The most widely used protocol in the internet	Used for voice over IP, streaming video, Gaming and Live Broadcasting
No packet loss during transmission	Faster and need Few Resources
Send packets in order	Packets don’t necessarily arrive in order
Slower and requires more resources	Allows missing packets, but the sender is unable to know whether a packet has been received.
Best suitable for high reliability and transmission time is relatively less critical	Best suitable for applications that need fast, efficient transmission, such as games.
TCP sends individual packets and is considered a reliable transport medium.	UDP sends messages, called datagrams and is considered a best-effort mode of communications.
TCP provides error and flow control, no such mechanisms are supported in UDP.	UDP is considered a connectionless protocol because it doesn't require a virtual circuit to be established before any data transfer occurs

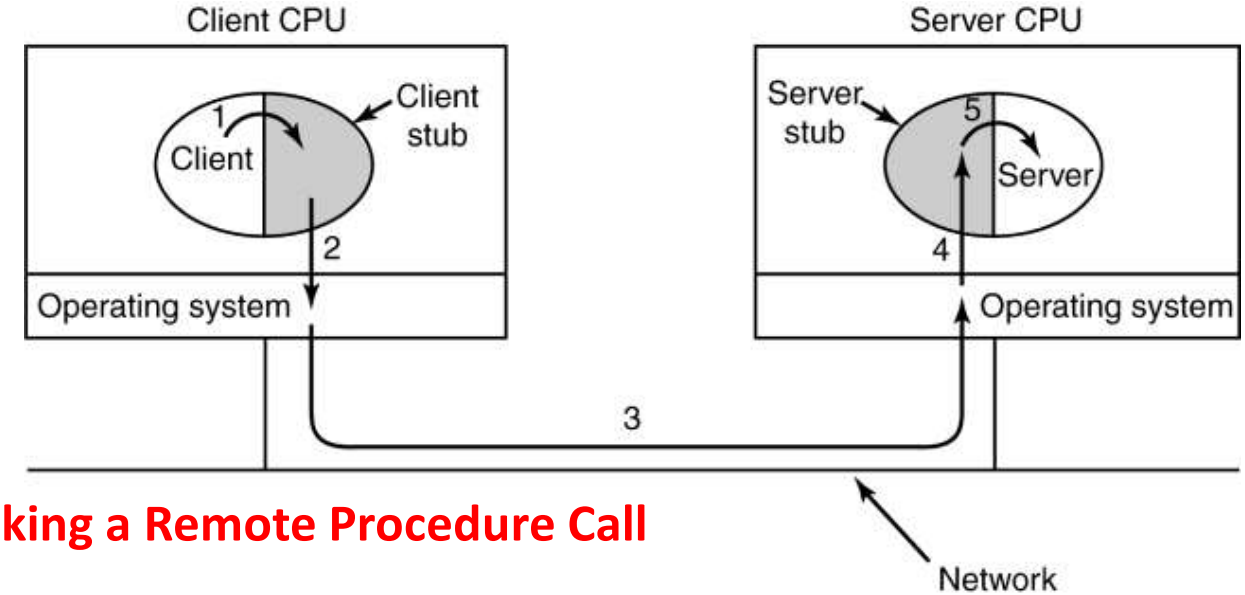
Remote Procedure Call

- In a certain sense, sending a message to a remote host and getting a reply back is a lot like making a function call in a programming language.
- Birrell and Nelson suggested was allowing programs to call procedures located on remote hosts.
- When a process on machine 1 calls a procedure on machine 2, the calling process on 1 is suspended and execution of the called procedure takes place on 2.
- Information can be transported from the caller to the callee in the parameters and can come back in the procedure result.
- No message passing is visible to the programmer. This technique is known as RPC (Remote Procedure Call) and has become the basis for many networking applications.

- In the simplest form, to call a remote procedure, the client program must be bound with a small library procedure, called the client stub, that represents the server procedure in the client's address space.
- Similarly, the server is bound with a procedure called the server stub.



The actual steps in making an RPC



Steps in making a Remote Procedure Call

Step 1 is the client calling the client stub. This call is a local procedure call, with the parameters pushed onto the stack in the normal way.

Step 2 is the client stub packing the parameters into a message and making a system call to send the message. Packing the parameters is called marshaling (assembling).

Step 3 is the kernel sending the message from the client machine to the server machine.

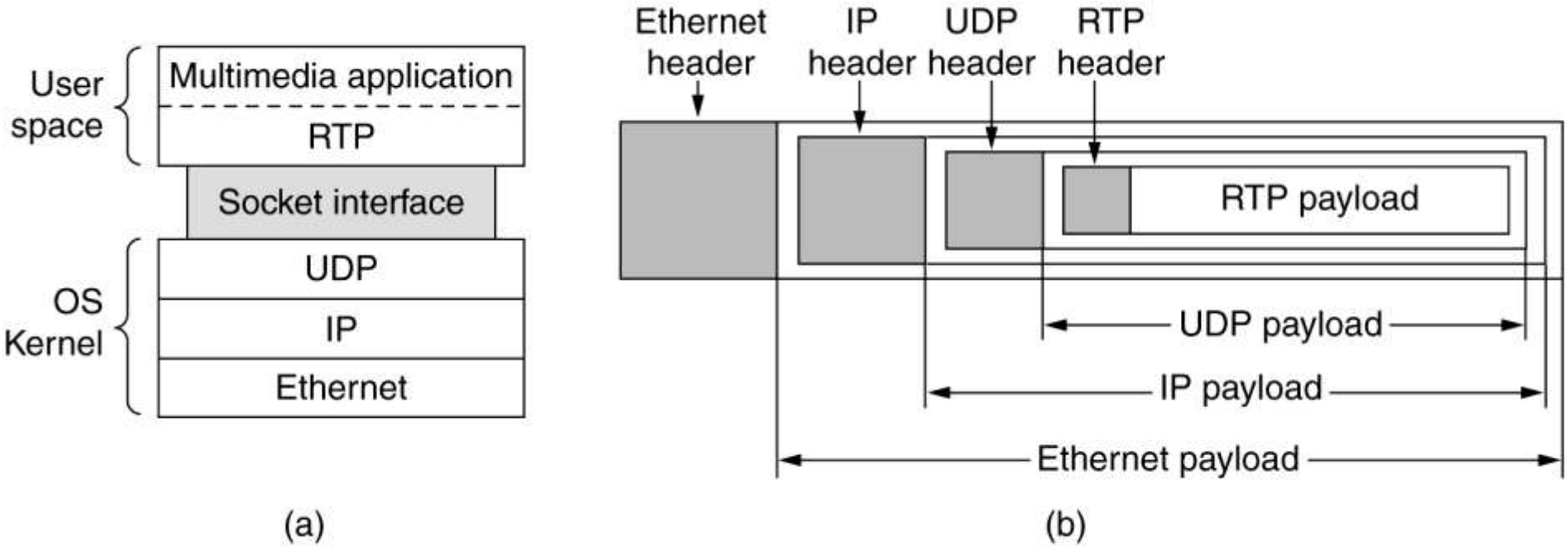
Step 4 is the kernel passing the incoming packet to the server stub.

Step 5 is the server stub calling the server procedure with the unmarshaled parameters. The reply traces the same path in the other direction.

The Real-Time Transport Protocol

- Client-server RPC is one area in which UDP is widely used.
- Another one is Real-time Multimedia Applications, as Internet radio, Internet telephony, music-on-demand, videoconferencing, video-on-demand, and other multimedia applications.
- Thus was RTP (Real-time Transport Protocol) born.
- RTP normally runs in user space over UDP (in the operating system).

The Real-Time Transport Protocol



(a) The position of RTP in the protocol stack.

(b) Packet nesting.

The two aspects of Real-time Transport Protocol: The first is the RTP protocol for transporting audio and video data in packets.

The second is the processing that takes place, mostly at the receiver, to play out the audio and video at the right time.

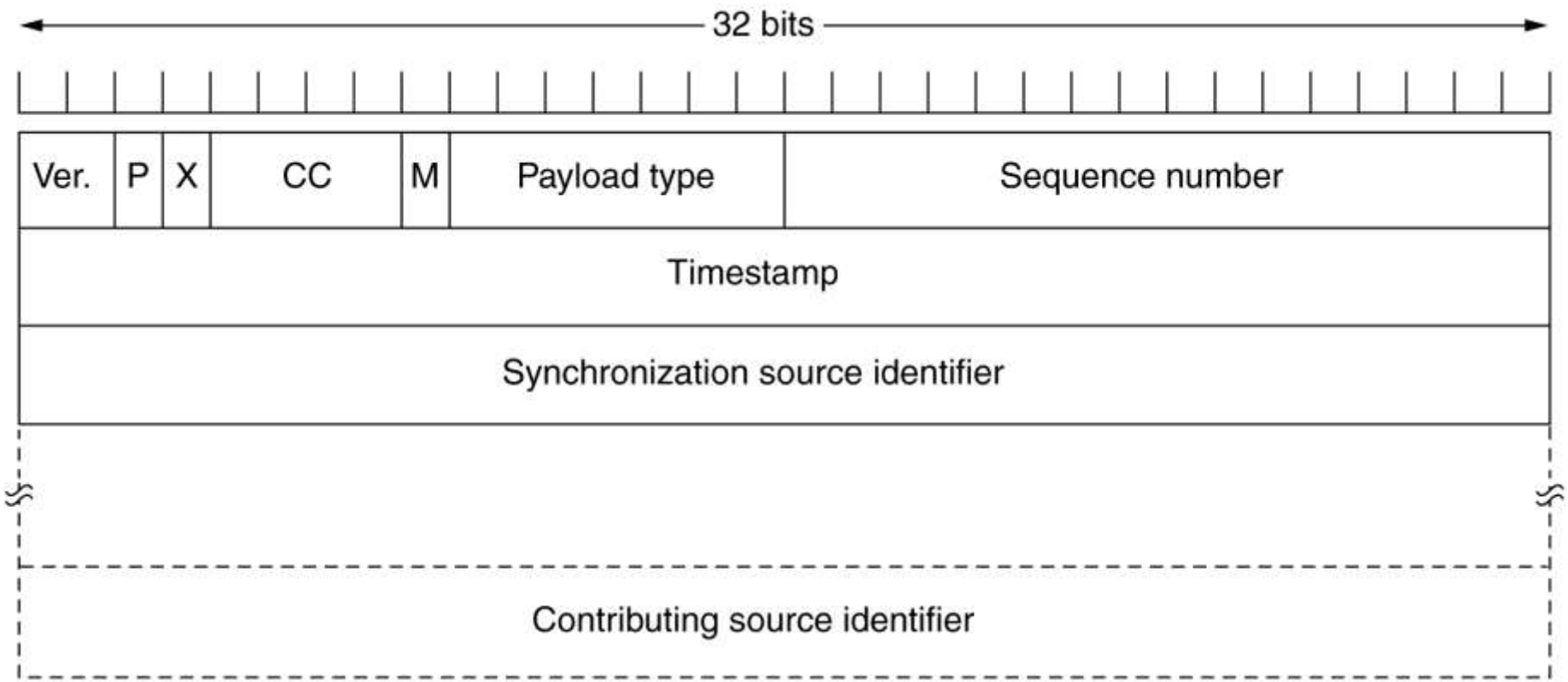
- As a consequence of this design, it is a little hard to say which layer RTP is in.

RTP—The Real-time Transport Protocol

- The basic function of RTP is to multiplex several real-time data streams onto a single stream of UDP packets.
- The UDP stream can be sent to a single destination (unicasting) or to multiple destinations (multicasting).
- Because RTP just uses normal UDP, its packets are not treated specially by the routers unless some normal IP quality-of-service features are enabled.
- In particular, there are no special guarantees about delivery, jitter, etc.
- **Jitter** is the **variation in time delay between when a signal is transmitted and when it's received over a network connection**. This is often caused by network congestion, poor hardware performance and not implementing packet prioritization.

- Each packet sent in an RTP stream is given a number one higher than its predecessor.
- This numbering allows the destination to determine if any packets are missing.
- If a packet is missing, the best action for the destination to take is to approximate the missing value by interpolation.
- Retransmission is not a practical option since the retransmitted packet would probably arrive too late to be useful.
- As a consequence, RTP has no flow control, no error control, no acknowledgements, and no mechanism to request retransmissions.

The Real-Time Transport Protocol Header



Ver-	Version
P-	Packet
X-	Extension Header
CC-	Contributing Sources
M-	Application-specific Mark

- It consists of three 32-bit words and potentially some extensions.
- The first word contains the Version field, which is already at 2.
- The P bit indicates that the packet has been padded to a multiple of 4 bytes.
- The last padding byte tells how many bytes were added.
- The X bit indicates that an Extension Header is present.
- The format and meaning of the extension header are not defined. The only thing that is defined is that the first word of the extension gives the length.

- The CC field tells how many Contributing Sources are present. It can be used to , from 0 to 15 .
- The M bit is an Application-specific Mark the start of a video frame, the start of a word in an audio channel, or something else that the application understands.
- The Payload type field tells which encoding algorithm has been used (e.g., uncompressed 8-bit audio, MP3, etc.).
- The Sequence number is just a counter that is incremented on each RTP packet sent. It is used to detect lost packets.

- The Timestamp is produced by the stream's source to note when the first sample in the packet was made. This value can help reduce jitter.
- The Synchronization source identifier tells which stream the packet belongs to. It is the method used to multiplex and demultiplex multiple data streams onto a single stream of UDP packets.
- The Contributing source identifiers, if any, are used when mixers are present in the studio. In that case, the mixer is the synchronizing source, and the streams being mixed are listed here.

The Internet Transport Protocols: TCP

Introduction to TCP (**Transmission Control Protocol)**

1. The TCP Service Model
2. The TCP Protocol
3. The TCP Segment Header
4. TCP Connection Establishment
5. TCP Connection Release
6. Sliding Window
7. TCP Timer Management
8. TCP Congestion Control

Introduction to TCP (Transmission Control Protocol)

- UDP is a simple protocol and it has some very important uses, such as client server interactions and multimedia, but for most Internet applications, reliable, sequenced delivery is needed.
- UDP cannot provide this, so another protocol is required. It is called **TCP** and is the main workhorse of the Internet.
- **TCP (Transmission Control Protocol)** was **specifically designed** to provide **a reliable end-to-end byte stream** over an **unreliable internetwork**.
- TCP was designed to **dynamically adapt to properties of the internetwork** and to be robust in the face of many kinds of failures.

1. The TCP Service Model

- TCP service is obtained by both the sender and the receiver creating end points, called **sockets.**
- Each socket has a socket number (address) consisting of the IP address of the host and a 16-bit number local to that host, called a **port.**
- A port is the **TCP** name for a **TSAP :** A **Transport Services Access Point** is an end-point for communication between the Transport layer and the Session layer in the OSI (Open Systems Interconnection) reference model.
- Each TSAP is an address that uniquely identifies a specific instantiation of a service.
- TSAPs are created by concatenating the node's Network Service Access Point (NSAP) with a transport identifier, and sometimes a packet and/or protocol type.

A socket may be used for multiple connections at the same time.

Connections are identified by the socket identifiers at both ends, that is, (*socket1, socket2). No* virtual circuit numbers or other identifiers are used.

The socket primitives for TCP

- Port numbers **below 1024** are reserved for standard services that can usually only be started by privileged users (e.g., root in UNIX systems).
- They are called **well-known ports.**

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

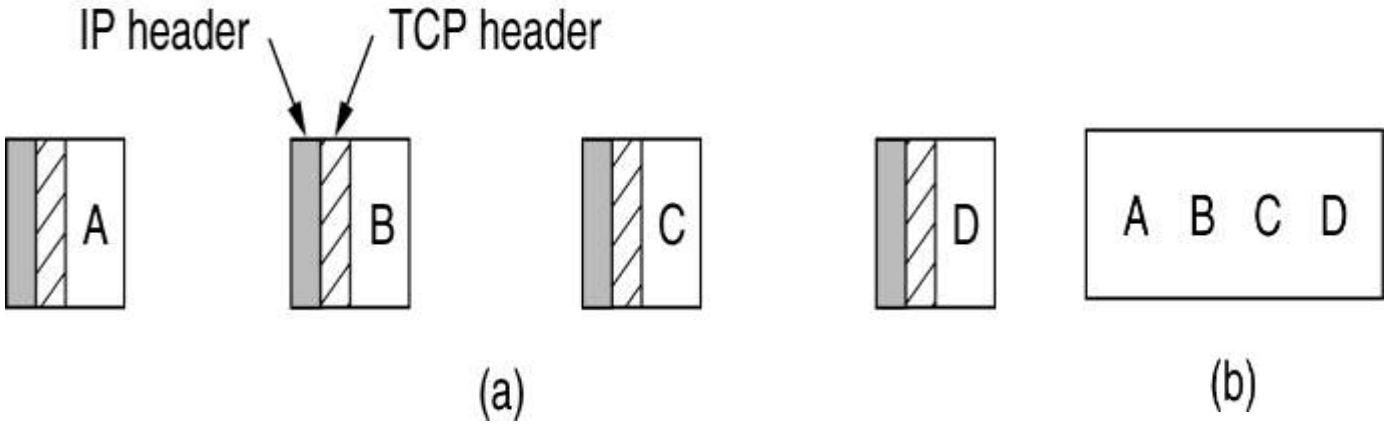
- **Secure Shell (SSH)** is a cryptographic network protocol for operating network services securely over an unsecured network
- **IMAP (Internet Message Access Protocol)** is a standard email protocol that stores email messages on a mail server, but allows the end user to view and manipulate the messages as though they were stored locally on the end user's computing device(s).
- **The Real Time Streaming Protocol (RTSP)** is a network control protocol designed for use in entertainment and communications systems to control streaming media servers. The protocol is used for establishing and controlling media sessions between end points.
- **The Internet Printing Protocol (IPP)** is a specialized Internet protocol for communication between client devices (computers, mobile phones, tablets, etc.) and printers

All TCP connections are full duplex and point-to-point.

- **Full duplex** means that traffic can go in both directions at the same time.
- **Point-to-point** means that each connection has exactly two end points.
- **TCP does not support Multicasting or Broadcasting.**
- A TCP connection is a **byte stream**, not a message stream.
- Message boundaries are not preserved end to end.

For example, if the sending process does four 512-byte writes to a TCP stream, these data may be delivered to the receiving process as four 512-byte chunks, two 1024-byte chunks, one 2048-byte chunk , or some other way.

The TCP Service Model



(a) Four 512-byte segments sent as separate IP datagrams.

(b) The 2048 bytes of data delivered to the application in a single READ CALL.

2. The TCP Protocol

- A key feature of TCP, is that every byte on a TCP connection has its own **32-bit sequence number**.
- The sending and receiving TCP entities exchange data in the form of segments.
- A **TCP segment** consists of a **fixed 20-byte header** (plus an optional part) followed by zero or more data bytes.
- The **TCP software decides** how big segments should be. It can accumulate data from several writes into one segment or can split data from one write over multiple segments.
- **Two Limits Restrict the Segment Size. First,** each segment, including the TCP header, must fit in the 65,515- byte IP payload**. Second,** each link has an **MTU (Maximum Transfer Unit).**
- Each segment must fit in the MTU at the sender and receiver so that it can be sent and received in a single, unfragmented packet. In practice, the MTU is generally 1500 bytes (the Ethernet payload size) and thus defines the upper bound on segment size.

- The basic protocol used by TCP entities is the sliding window protocol with a dynamic window size. When a sender transmits a segment, it also starts a timer.
- When the segment arrives at the destination, the receiving TCP entity sends back a segment bearing an acknowledgement number equal to the next sequence number it expects to receive and the remaining window size.
- If the sender's timer goes off before the acknowledgement is received, the sender transmits the segment again.

The advantages of TCP/IP protocol suite are

- It is an industry–standard model that can be effectively deployed in practical networking problems.
- It is interoperable, i.e., it allows cross-platform communications among heterogeneous networks.
- It is an open protocol suite. It is not owned by any particular institute and so can be used by any individual or organization.
- It is a scalable, client-server architecture. This allows networks to be added without disrupting the current services.
- It assigns an IP address to each computer on the network, thus making each device to be identifiable over the network. It assigns each site a domain name. It provides name and address resolution services.

The disadvantages of the TCP/IP model are

- It is not generic in nature. So, it fails to represent any protocol stack other than the TCP/IP suite. For example, it cannot describe the Bluetooth connection.
- It does not clearly separate the concepts of services, interfaces, and protocols. So, it is not suitable to describe new technologies in new networks.
- It does not distinguish between the data link and the physical layers, which has very different functionalities. The data link layer should concern with the transmission of frames. On the other hand, the physical layer should lay down the physical characteristics of transmission. A proper model should segregate the two layers.
- It was originally designed and implemented for wide area networks. It is not optimized for small networks like LAN (local area network) and PAN (personal area network).
- Among its suite of protocols, TCP and IP were carefully designed and well implemented. Some of the other protocols were developed ad hoc and so proved to be unsuitable in long run. However, due to the popularity of the model, these protocols are being used even 30–40 years after their introduction.

UNIT-IV: TRANSPORT LAYER

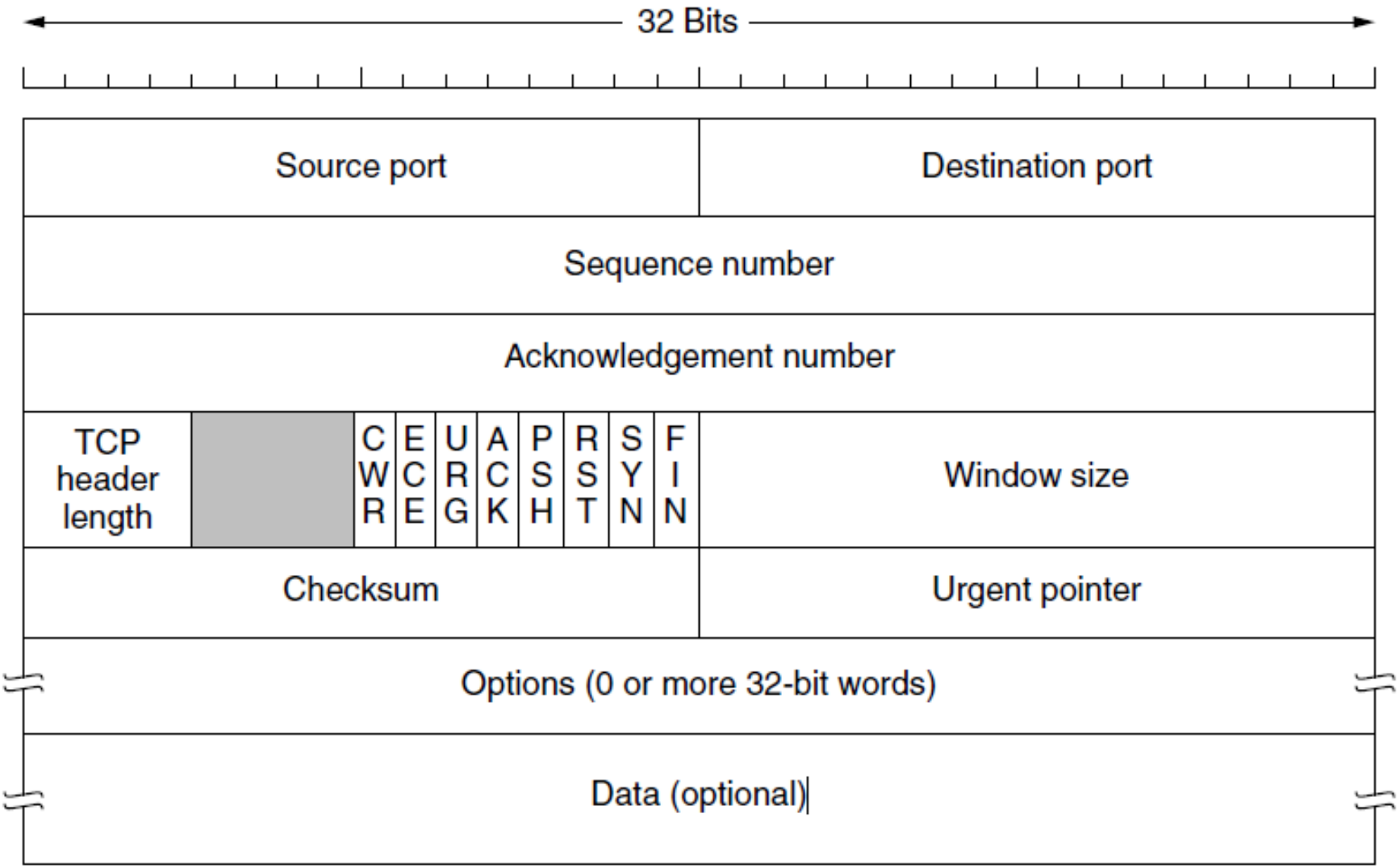
UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. Timer management.
8. Congestion control.

3. The TCP Segment Header



1. The **Source port** and **Destination port fields** identify the local end points of the connection. A TCP port plus its host's IP address forms a 48-bit unique end point.

   The connection identifier is called a **5 tuple** because it consists of five pieces of information**:** The protocol (TCP), Source IP and Source Port, and Destination IP and Destination Port.

2. The **Sequence number** and **Acknowledgement number fields** perform their usual functions. Note that the latter specifies the next in-order byte expected, not the last byte correctly received. It is a cumulative acknowledgement because it summarizes the received data with a single number. It does not go beyond lost data. Both are 32 bits because every byte of data is numbered in a TCP stream.

3. The **TCP header length** tells how many 32-bit words are contained in the TCP header.

4. A **4-bit field** that is not used.
5. **Next comes to eight 1-bit flags:**
   i. CWR (Congestion Window Reduced)
   ii. ECN (Explicit Congestion Notification)
   iii. URG (Urgent pointer)
   iv. ACK (Acknowledgement number)
   v. PSH (PUSHed data)
   vi. RST (Reset a connection)
   vii. The SYN bit is used to establish connections
   viii. The FIN bit is used to release a connection

- **ECE (Explicit Congestion Notification-Echo)** is set to signal an ECN-Echo to a TCP sender to tell it to slow down when the TCP receiver gets a congestion indication from the network.

- **CWR (Congestion Window Reduced)** is set to signal Congestion Window Reduced from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the **ECN-Echo.**
- **URG** is set to 1 if the **Urgent pointer** is in use. The Urgent pointer is used to indicate a byte offset from the current sequence number at which urgent data are to be found. This facility is in lieu of interrupt messages.
- The **ACK bit** is set to 1 to indicate that the **Acknowledgement number** is valid. This is the case for nearly all packets. If ACK is 0, the segment does not contain an acknowledgement, so the Acknowledgement number field is ignored.
- The **PSH bit** indicates **PUSHed data**. The receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received (which it might otherwise do for efficiency).
- The **RST bit** is used to suddenly **reset a connection** that has become confused due to a host crash or some other reason. It is also used to reject an invalid segment or refuse an attempt to open a connection. In general, if you get a segment with the RST bit on, you have a problem on your hands.

- The **SYN bit** is used to **establish connections**. The connection request has SYN = 1 and ACK = 0 to indicate that the piggyback acknowledgement field is not in use. The connection reply does bear an acknowledgement, however, so it has SYN = 1 and ACK = 1. In essence, the SYN bit is used to denote both CONNECTION REQUEST and CONNECTION ACCEPTED, with the ACK bit used to distinguish between those two possibilities.
- The **FIN bit** is used to **release a connection**. It specifies that the sender has no more data to transmit. However, after closing a connection, the closing process may continue to receive data indefinitely. Both SYN and FIN segments have sequence numbers and are thus guaranteed to be processed in the correct order.

6. Flow control in TCP is handled using a variable-sized sliding window. The **Window size field** tells how many bytes may be sent starting at the byte acknowledged. A Window size field of 0 is legal and Ack. number is 1.

7. A ***Checksum*** *is also provided for extra reliability. It checksums the header,* the data, and a conceptual pseudoheader in exactly the same way as UDP, except that the pseudoheader has the protocol number for TCP (6) and the checksum is mandatory.

8. The **Options field** provides a way to **add extra facilities** not covered by the regular header.

   The **options** are of **variable length**, fill a multiple of 32 bits by using padding with zeros, and may extend to 40 bytes to accommodate the longest TCP header that can be specified. Some options are carried when a **connection is established** to negotiate or inform the other side of capabilities. Other options are carried on packets during the lifetime of the connection. Each option has a **Type-Length-Value encoding.** A widely used option is the one that allows each host to specify the **MSS (Maximum Segment Size). The window scale option** allows the sender and receiver to negotiate a window scale factor at the start of a connection.

- **The timestamp option** carries a timestamp sent by the sender and echoed by the receiver.
- Finally, the **SACK (Selective ACKnowledgement**) option lets a receiver tell a sender the ranges of sequence numbers that it has received. It supplements the Acknowledgement number and is used after a packet has been lost but subsequent (or duplicate) data has arrived.

UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

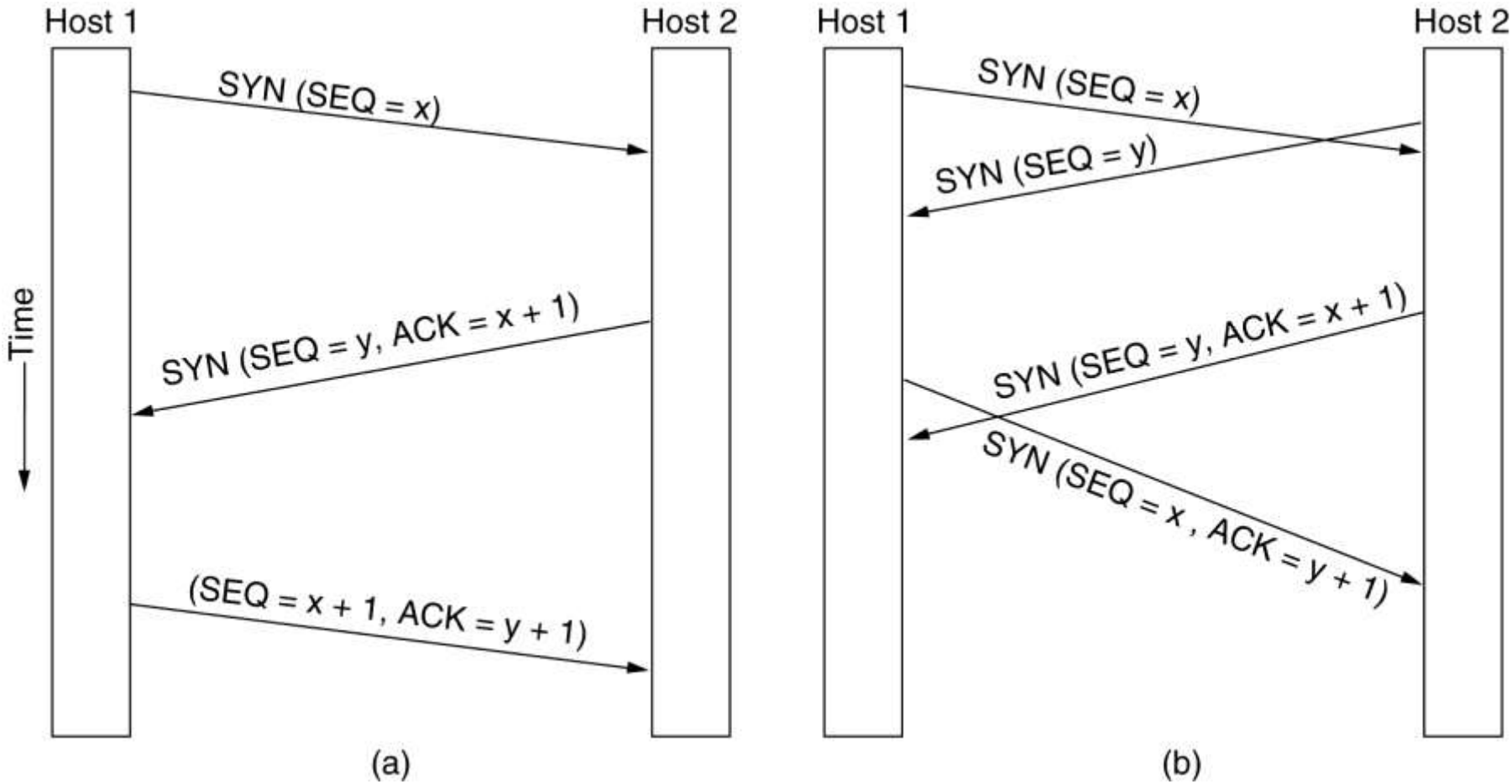
TCP

1. service model.
2. Protocol.
3. Segment header.
4. TCP Connection Establishment .
5. TCP Connection Release .
6. TCP Sliding Window .
7. TCP Timer Management.
8. TCP Congestion Control.

4. TCP Connection Establishment

- Connections are established in TCP by means of the three-way handshake .
- To establish a connection, one side, say, the **server**, passively waits for an incoming connection by executing the LISTEN and ACCEPT primitives, either specifying a specific source or nobody in particular.
- The **other side, say, the client**, executes a CONNECT primitive, specifying the IP address and port to which it wants to connect, the maximum TCP segment size it is willing to accept, and optionally some user data (e.g., a password). The CONNECT primitive sends a TCP segment with the SYN bit on and ACK bit off and waits for a response.
- When this segment arrives at the destination, the TCP entity there checks to see if there is a process that has done a LISTEN on the port given in the Destination port field. If not, it sends a reply with the RST bit on to reject the connection

TCP Connection Establishment



(a) TCP connection establishment in the normal case.

(b) Simultaneous connection establishment on both sides.

TCP Connection Management Modeling

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. Timer management.
8. Congestion control.

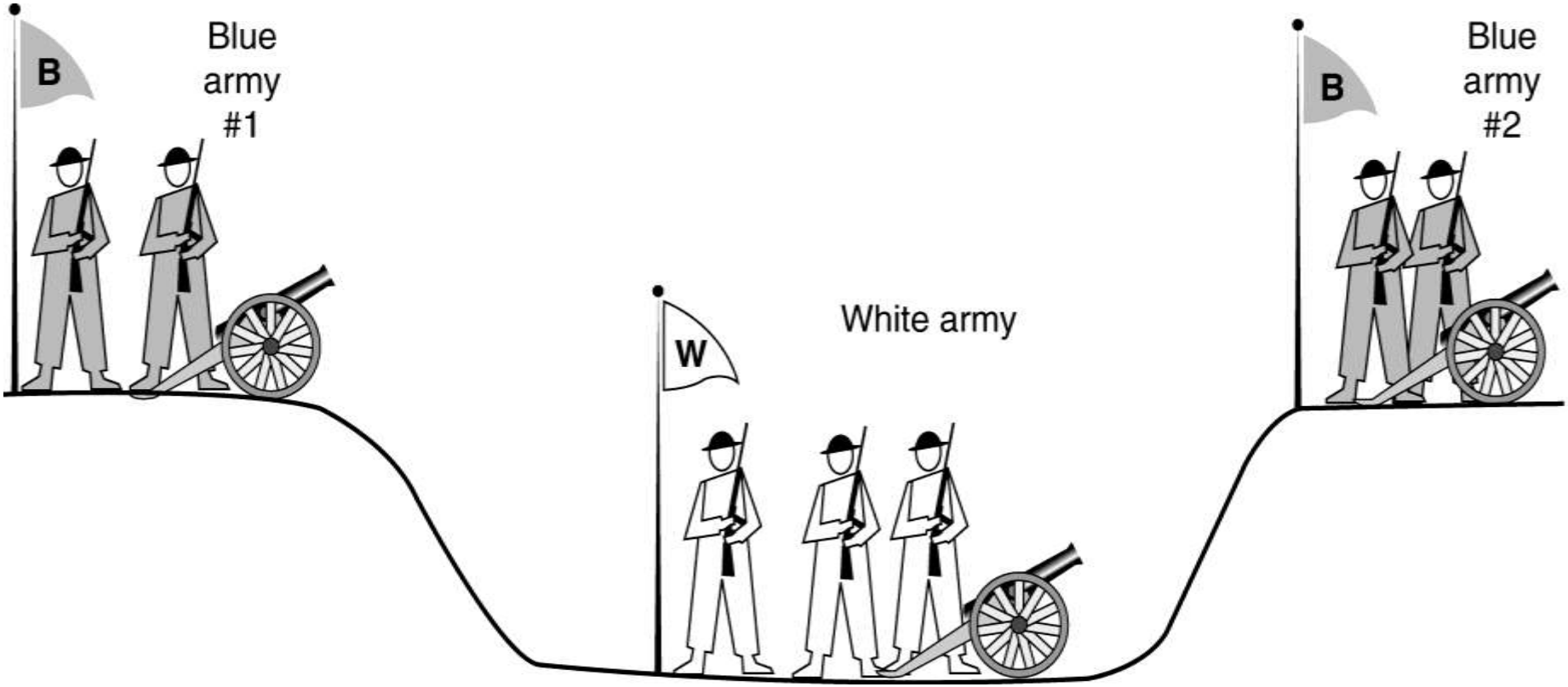
5. TCP Connection Release

- Each simplex connection is released independently of its sibling.
- To release a connection, either party can send a TCP segment with the FIN bit set, which means that it has no more data to transmit.
- When the FIN is acknowledged, that direction is shut down for new data.
- Data may continue to flow indefinitely in the other direction, however.
- When both directions have been shut down, the connection is released.
- To avoid the two-army problem, timers are used.
- If a response to a FIN is not forthcoming within two maximum packet lifetimes, the sender of the FIN releases the connection.
- The other side will eventually notice that nobody seems to be listening to it any more and will time out as well.

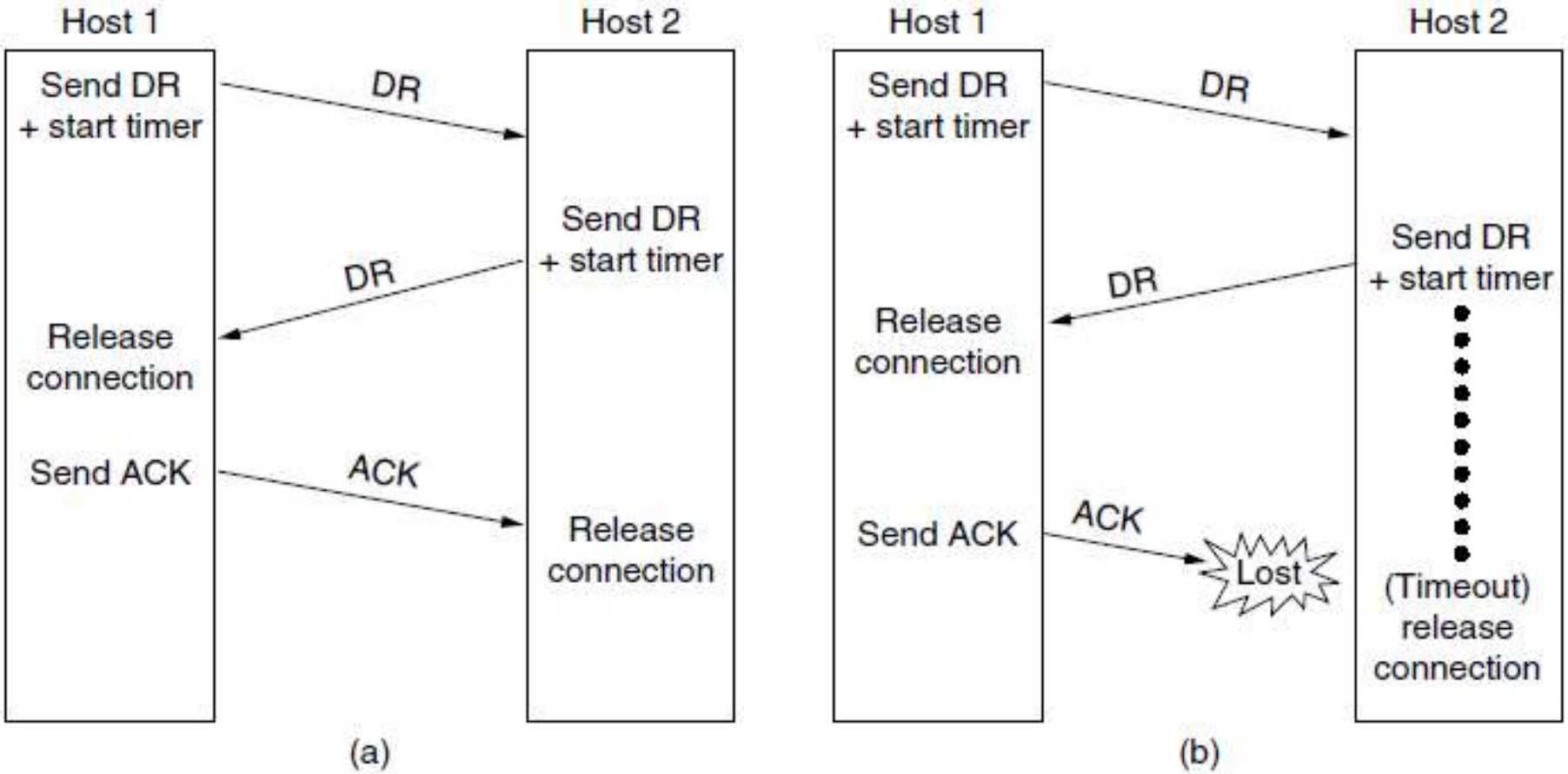
Connection Release: The two-army problem

Unfortunately, this protocol does not always work.

The two-army problem illustrates four scenarios of releasing using a three-way handshake. While this protocol is not reliable, it is usually adequate.



DR (DISCONNECTION REQUEST)



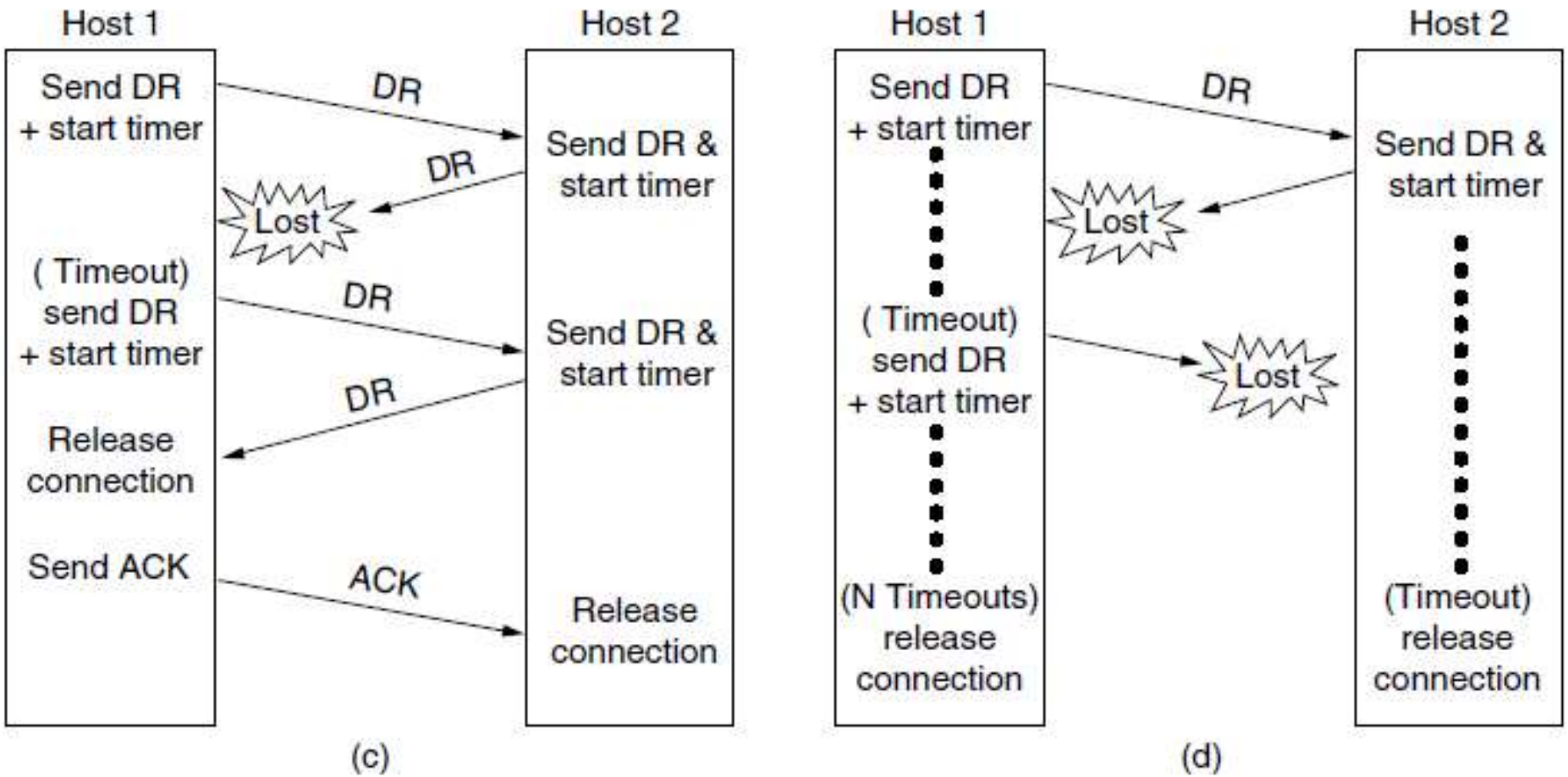
Four protocol scenarios for releasing a connection.

(a) Normal case of three-way handshake.

(b) Final ACK lost.

(c) Response lost.

(d) Response lost and subsequent DRs lost.



Four protocol scenarios for releasing a connection.

(a) Normal case of three-way handshake.

(b) Final ACK lost.

(c) Response lost.

(d) Response lost and subsequent DRs lost.

UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

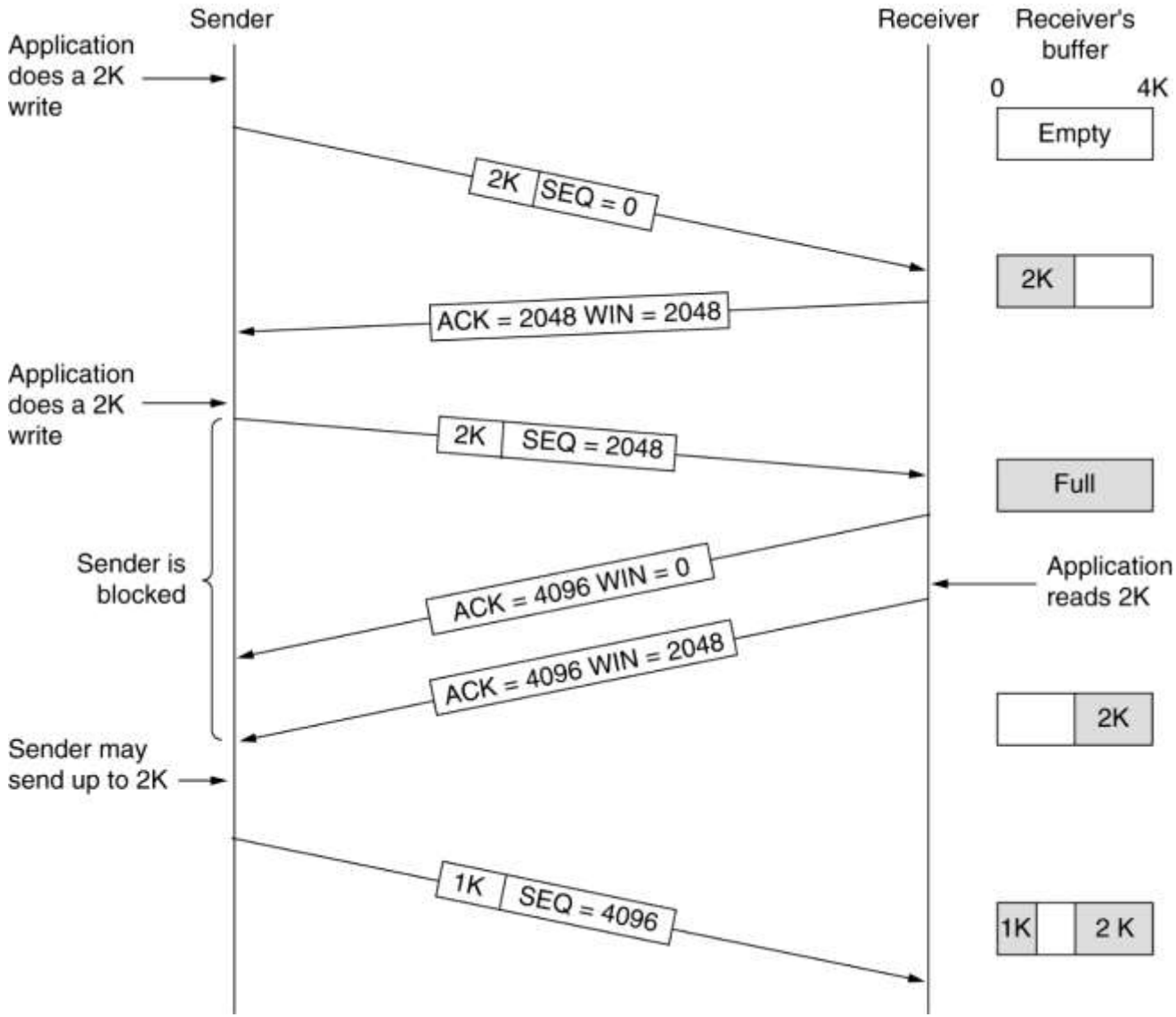
TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. Timer management.
8. Congestion control.

6. TCP Sliding Window

1. A frame carries an **Error-Detecting Code** (e.g., a CRC or checksum) that is used to check if the information was correctly received.
2. A frame carries a sequence number to identify itself and is retransmitted by the sender until it receives an acknowledgement of successful receipt from the receiver. This is called **ARQ (Automatic Repeat reQuest).**
3. There is a maximum number of frames that the sender will allow to be outstanding at any time, pausing if the receiver is not acknowledging frames quickly enough. If this maximum is one packet the protocol is called **stop-and-wait**. **Larger windows enable pipelining and** improve performance on long, fast links.
4. The **sliding window protocol** **combines these features and is also** used to **support bidirectional data transfer**.

TCP Sliding Window: TCP Transmission Policy



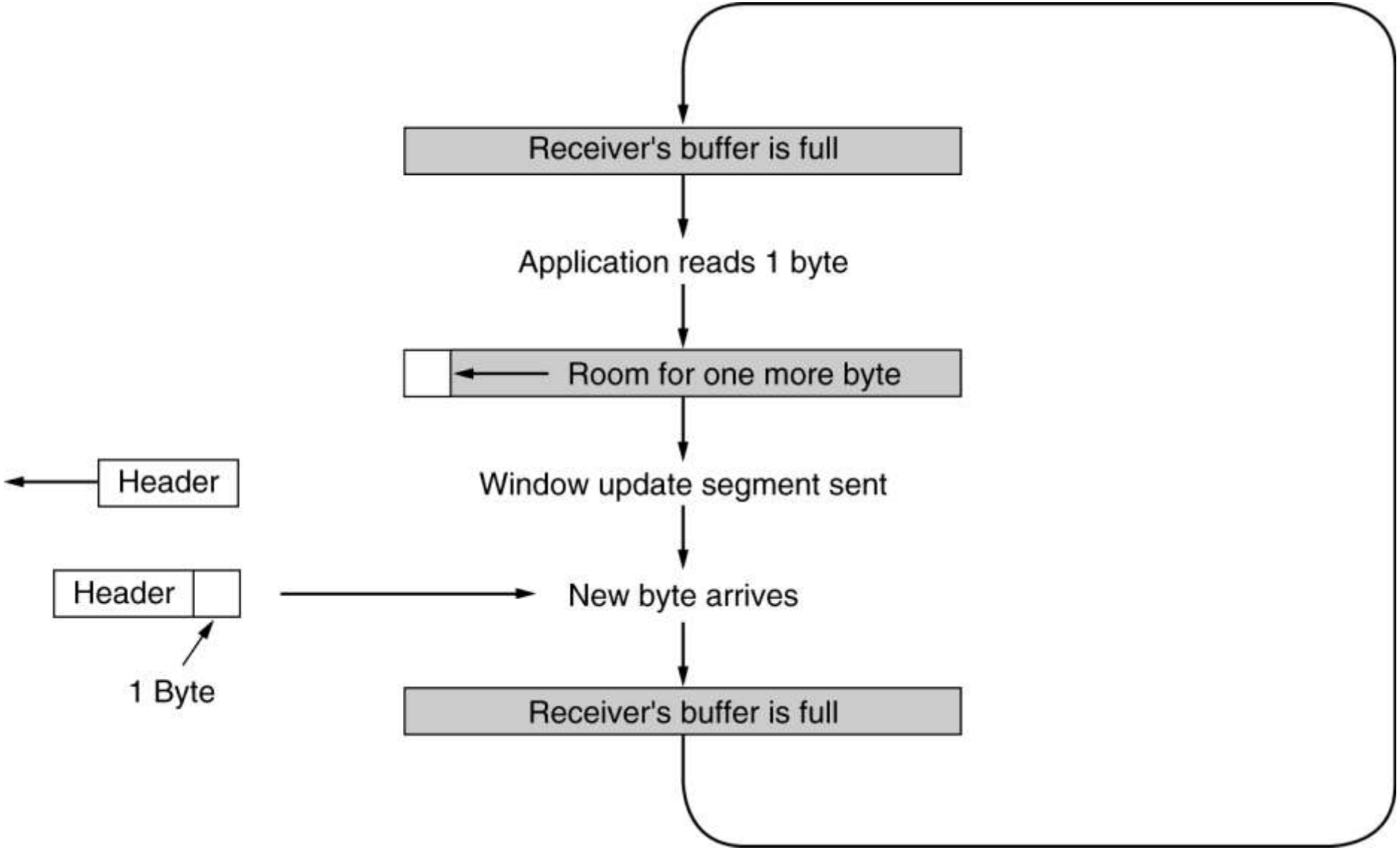
Window management in TCP.

- When the window is 0, the sender may not normally send segments, with two exceptions. First, urgent data may be sent, for example, to allow the user to kill the process running on the remote machine. Second, the sender may send a 1-byte segment to force the receiver to reannounce the next byte expected and the window size. This packet is called a **window probe**.
- The TCP standard explicitly provides this option to prevent deadlock if a window update ever gets lost.
- One approach that many TCP implementations use to optimize Bandwidth situation is called **delayed acknowledgements**.
- The idea is to delay acknowledgements and window updates for up to 500 msec in the hope of acquiring some data on which to catch a free ride.
- Although delayed acknowledgements reduce the load placed on the network by the receiver, a sender that sends multiple short packets (e.g., 41-byte packets containing 1 byte of data) is still operating inefficiently. A way to reduce this usage is known as **Nagle's algorithm** (Nagle, 1984).

- **What Nagle suggested is simple:** when data come into the sender in small pieces, just send the first piece and buffer all the rest until the first piece is acknowledged.
- Then send all the buffered data in one TCP segment and start buffering again until the next segment is acknowledged.
- That is, only one short packet can be outstanding at any time.
- If many pieces of data are sent by the application in one round-trip time, Nagle's algorithm will put the many pieces in one segment, greatly reducing the bandwidth used.
- The algorithm additionally says that a new segment should be sent if enough data have trickled in to fill a maximum segment.
- **A more subtle problem is that Nagle's algorithm** can sometimes interact with delayed acknowledgements to cause a temporary deadlock: the receiver waits for data on which to piggyback an acknowledgement, and the sender waits on the acknowledgement to send more data. This interaction can delay the downloads of Web pages. Because of these problems, Nagle's algorithm can be disabled

- Another problem that can degrade TCP performance is the **Silly window syndrome** (Clark, 1982). This problem occurs when data are passed to the sending TCP entity in large blocks, but an interactive application on the receiving side reads data only 1 byte at a time.
- Clark's solution is to prevent the receiver from sending a window update for 1 byte.
- Instead it is forced to wait until it has a decent amount of space available and advertise that instead.
- Specifically, the receiver should not send a window update until it can handle the maximum segment size it advertised when the connection was established or until its buffer is half empty, whichever is smaller

- Nagle's algorithm and Clark's solution to the silly window syndrome are complementary.
- **Nagle** was trying to solve the problem caused by the **sending application** delivering data to TCP a byte at a time.
- **Clark** was trying to solve the problem of the **receiving application** sucking the data up from TCP a byte at a time. Both solutions are valid and can work together.
- The goal is for the sender not to send small segments and the receiver not to ask for them.
- Acknowledgements can be sent only when all the data up to the byte acknowledged have been received. This is called a **cumulative acknowledgement**.



Silly window syndrome.

UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

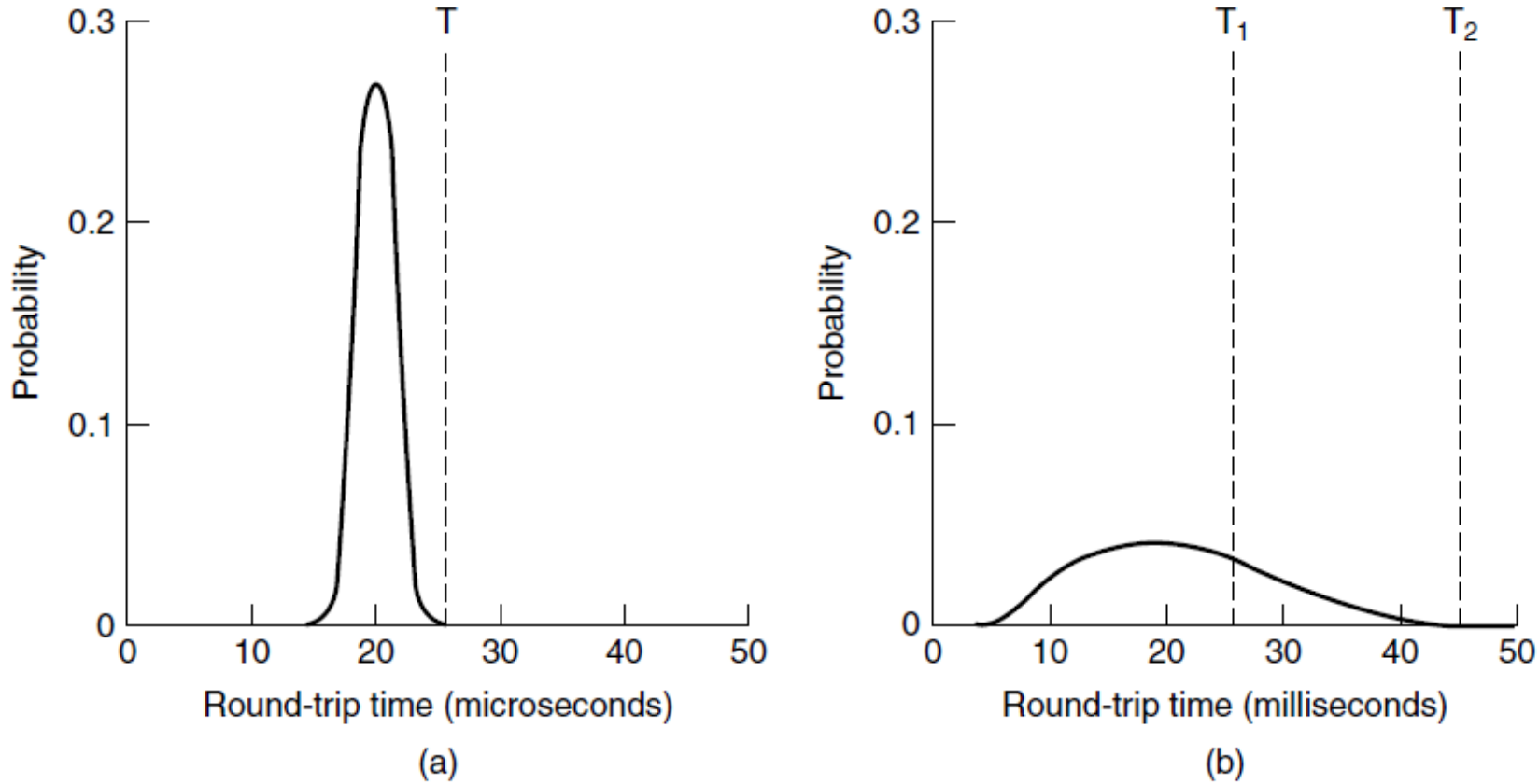
TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. **Timer management.**
8. Congestion control.

7. TCP Timer Management

- TCP uses multiple timers to do its work.
  - Retransmission timer
  - Persistence timer
  - Keepalive timer
- The most important of these is the **RTO (Retransmission TimeOut).** When a segment is sent, a retransmission timer is started.
- If the segment is acknowledged before the timer expires, the timer is stopped.
- If, on the other hand, the timer goes off before the acknowledgement comes in, the segment is retransmitted.
- The question that arises is: **How long should the timeout interval be?**

- **As Shown in Figure (a) Probability density of acknowledgement arrival times in the data link layer.**
- Since acknowledgements are rarely delayed in the data link layer (due to lack of congestion), the absence of an acknowledgement at the expected time generally means either the frame or the acknowledgement has been lost.
- If the timeout is set too short, say, T1 in Figure (b), unnecessary retransmissions will occur, clogging the Internet with useless packets.
- If it is set too long (e.g., T2), performance will suffer due to the long retransmission delay whenever a packet is lost. Furthermore, the **mean** and **variance** of the acknowledgement arrival distribution can change rapidly within a few seconds as congestion builds up or is resolved.



(a) Probability density of acknowledgement arrival times in the data link layer.
(b) Probability density of acknowledgement arrival times for TCP.

- The solution is to use a dynamic algorithm that constantly adapts the timeout interval, based on continuous measurements of network performance.
- The algorithm generally used by TCP is due to Jacobson (1988) and works as follows. For each connection, TCP maintains a variable, **SRTT (Smoothed Round-Trip Time),** that is the best current estimate of the round-trip time to the destination in question.
- When a segment is sent, a timer is started, both to see how long the acknowledgement the acknowledgement gets back before the timer expires, TCP measures how long the acknowledgement took, say, R. It then updates SRTT according to the formula

$$\mathbf{SRTT = \alpha\ SRTT + (1 - \alpha)\ R}$$

- where α is a smoothing factor that determines how quickly the old values are forgotten.
- Typically, α = 7/8. This kind of formula is an **EWMA (Exponentially Weighted Moving Average)** or low-pass filter that discards noise in the samples.

- **RTTVAR** (Round-Trip Time VARiation) that is updated using the formula

$$\mathbf{RTTVAR = \beta\ RTTVAR + (1 - \beta)\ |SRTT - R\ |}$$

- This is an EWMA as before, and typically $\beta = 3/4$.
- The retransmission timeout, RTO, is set to be

$$\mathbf{RTO = SRTT + 4 \times RTTVAR}$$

Persistence timer

- Phil Karn made a simple proposal: do not update estimates on any segments that have been retransmitted. Additionally, the timeout is doubled on each successive retransmission until the segments get through the first time. This fix is called **Karn's algorithm (Karn and Partridge,** 1987). Most TCP implementations use it.
- A second timer is the **persistence timer.** It is designed to prevent the following deadlock.
- The receiver sends an acknowledgement with a window size of 0, telling the sender to wait. Later, the receiver updates the window, but the packet with the update is lost. Now both the sender and the receiver are waiting for each other to do something.
- When the persistence timer goes off, the sender transmits a probe to the receiver. The response to the probe gives the window size. If it is still zero, the persistence timer is set again and the cycle repeats. If it is nonzero, data can now be sent.

Keepalive timer

- A third timer that some implementations use is the keepalive timer.
- When a connection has been idle for a long time, the keepalive timer may go off to cause one side to check whether the other side is still there.
- If it fails to respond, the connection is terminated.
- This feature is controversial because it adds overhead and may terminate an otherwise healthy connection due to a transient network partition.
- The last timer used on each TCP connection is the one used in the TIME WAIT state while closing.
- It runs for twice the maximum packet lifetime to make sure that when a connection is closed, all packets created by it have died off.

UNIT-IV: TRANSPORT LAYER

UDP

1. Segment header
2. Remote Procedure Call (RPC).
3. Real-time Transport Protocols (RTP).

TCP

1. service model.
2. Protocol.
3. Segment header.
4. Connection establishment .
5. Connection release .
6. Sliding window .
7. **Timer management.**
8. Congestion control.

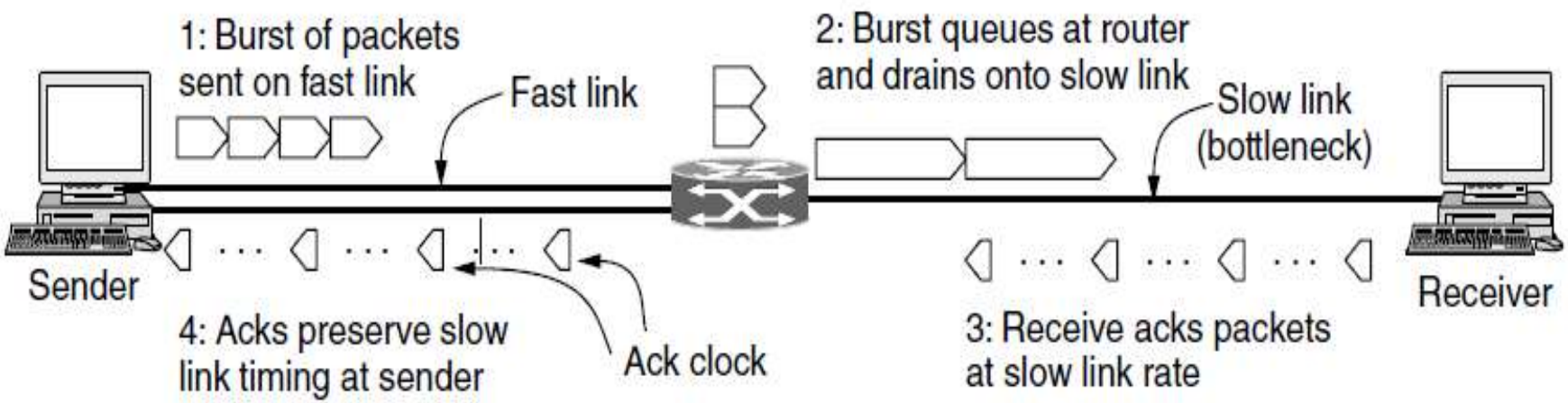
8. TCP Congestion Control

- When the load offered to any network is more than it can handle, congestion builds up.
- The Internet is no exception. The network layer detects congestion when queues grow large at routers and tries to manage it, if only by dropping packets.
- It is up to the transport layer to receive congestion feedback from the network layer and slow down the rate of traffic that it is sending into the network.
- In the Internet, TCP plays the main role in controlling congestion, as well as the main role in reliable transport. That is why it is such a special protocol.

- A transport protocol using an **AIMD (Additive Increase Multiplicative Decrease)** control law in response to binary congestion signals from the network would converge to a fair and efficient bandwidth allocation.
- TCP congestion control is based on implementing this approach using a window and with packet loss as the binary signal.
- To do so, TCP maintains a **congestion window** whose size is the number of bytes the sender may have in the network at any time.
- The corresponding rate is the window size divided by the round-trip time of the connection.
- TCP adjusts the size of the window according to the AIMD rule.

- Before discussing how TCP reacts to congestion, let us first describe what it does to try to prevent congestion from occurring in the first place.
- When a connection is established, a suitable window size has to be chosen.
- The receiver can specify a window based on its buffer size.
- If the sender sticks to this window size, problems will not occur due to buffer overflow at the receiving end, but they may still occur due to internal congestion within the network.

- The below figure shows what happens when a sender on a fast network (the 1-Gbps link) sends a small burst of four packets to a receiver on a slow network (the 1- Mbps link) that is the bottleneck or slowest part of the path.
- Initially the four packets travel over the link as quickly as they can be sent by the sender.
- At the router, they are queued while being sent because it takes longer to send a packet over the slow link than to receive the next packet over the fast link.
- But the queue is not large because only a small number of packets were sent at once. Note the increased length of the packets on the slow link.
- The same packet, of 1 KB say, is now longer because it takes more time to send it on a slow link than on a fast one.



A burst of packets from a sender and the returning ack clock.

- the acknowledgements return to the sender at about the rate that packets can be sent over the slowest link in the path. This is precisely the rate that the sender wants to use.
- If it injects new packets into the network at this rate, they will be sent as fast as the slow link permits, but they will not queue up and congest any router along the path.
- This timing is known as an **ack clock**. It is an essential part of TCP. By using an ack clock, TCP smoothes out traffic and avoids unnecessary queues at routers.
- A **second consideration is that the AIMD rule** will take a very long time to reach a good operating point on fast networks if the congestion window is started from a small size. It would **cause congestion if used all at once.**
- The solution Jacobson chose to handle both of these considerations is a mix of linear and multiplicative increase.

- When a connection is established, the sender initializes the congestion window to a small initial value of at most four segments; and the use of four segments is an increase from an earlier initial value of one segment based on experience.
- The sender then sends the initial window. The packets will take a round-trip time to be acknowledged. For each segment that is acknowledged before the retransmission timer goes off.
- The congestion window is doubling every **Round Trip Time.**
- The **slow start algorithm,** but it is not slow at all—it is **exponential growth**—except in comparison to the previous algorithm that let an entire flow control window be sent all at once.
- In the first **round-trip time**, the sender injects one packet into the network (and the receiver receives one packet). Two packets are sent in the next round-trip time, then four packets in the third round-trip time.
- **Slow-start works well** over a range of link speeds and round-trip times, and uses an ack clock to match the rate of sender transmissions to the network path.

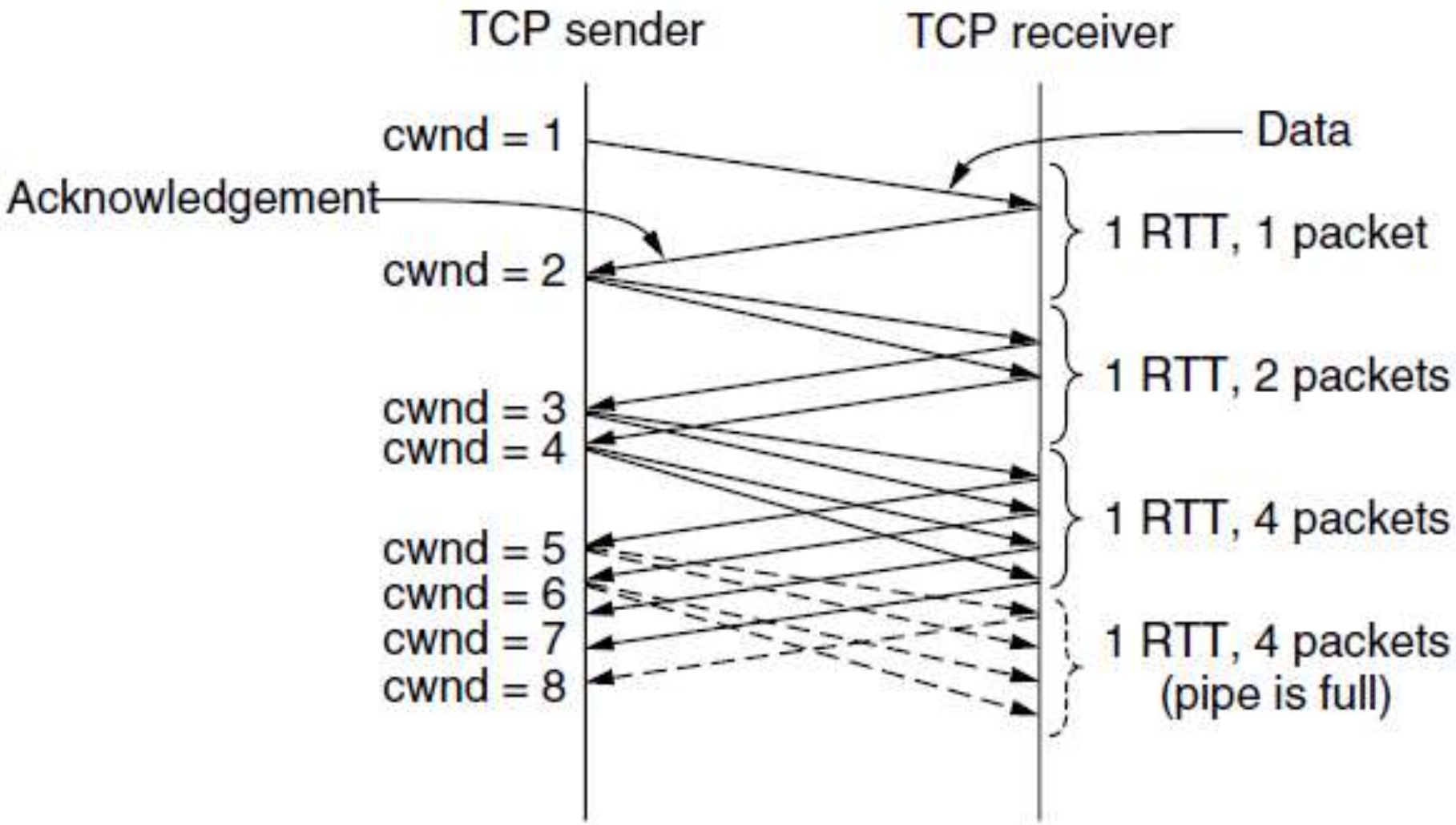
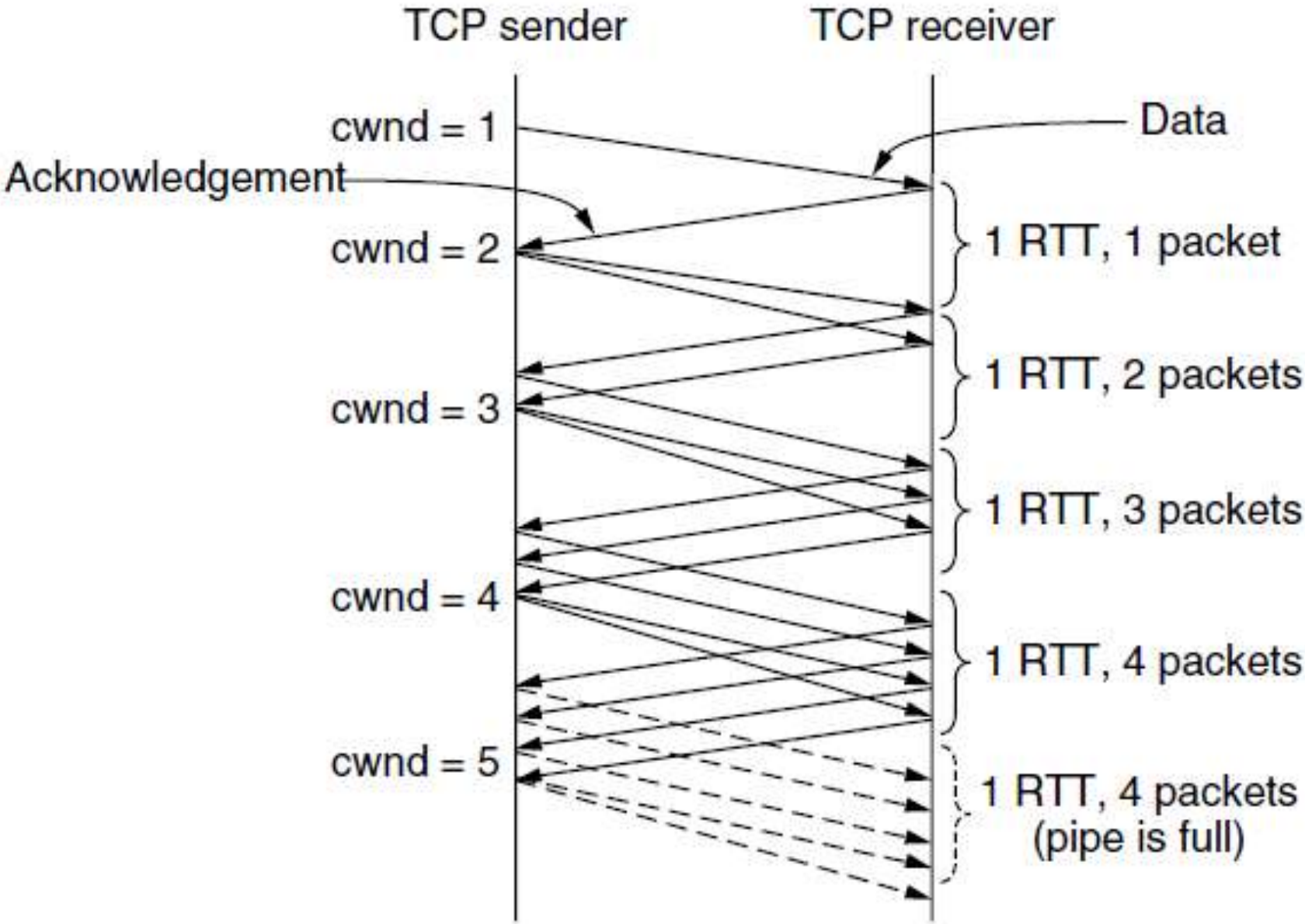


Fig. 6-44 Slow start algorithm from an initial congestion window of one segment.

- **Slow Start causes exponential growth**, eventually (and sooner rather than later) it will send too many packets into the network too quickly. When this happens, queues will build up in the network.
- When the queues are full, one or more packets will be lost. After this happens, the TCP sender will time out when an acknowledgement fails to arrive in time.
- There is evidence of slow start growing too fast in Fig. 6-44. After three RTTs, four packets are in the network. These four packets take an entire RTT to arrive at the receiver.
- Additional packets placed into the network by the sender will build up in router queues, since they cannot be delivered to the receiver quickly enough.
- **Congestion and packet loss will occur soon.**
- To keep **slow start under control**, the sender keeps a threshold for the connection called the **slow start threshold.**

- To keep slow start under control, the sender keeps a threshold for the connection called the **slow start threshold**. Initially this value is set arbitrarily high, to the size of the flow control window, so that it will not limit the connection.
- TCP keeps increasing the congestion window in slow start until a timeout occurs or the congestion window exceeds the threshold.
- Whenever a packet loss is detected, for example, by a timeout, the slow start threshold is set to be half of the congestion window and the entire process is restarted.
- Whenever the slow start threshold is crossed, TCP switches from slow start to additive increase.
- Call the congestion window **cwnd** and the maximum segment size **MSS**.
- A common approximation is to increase cwnd by **(MSS × MSS)/cwnd** for each of the **cwnd /MSS packets** that may be acknowledged. This increase does not need to be fast.
- The whole idea is for a TCP connection to spend a lot of time with its congestion window close to the optimum value—not so small that throughput will be low, and not so large that congestion will occur.
- Additive increase is shown in below figure, for the same situation as slow start. At the end of every RTT, the sender's congestion window has grown enough that it can inject an additional packet into the network.



Additive increase from an initial congestion window of one segment.

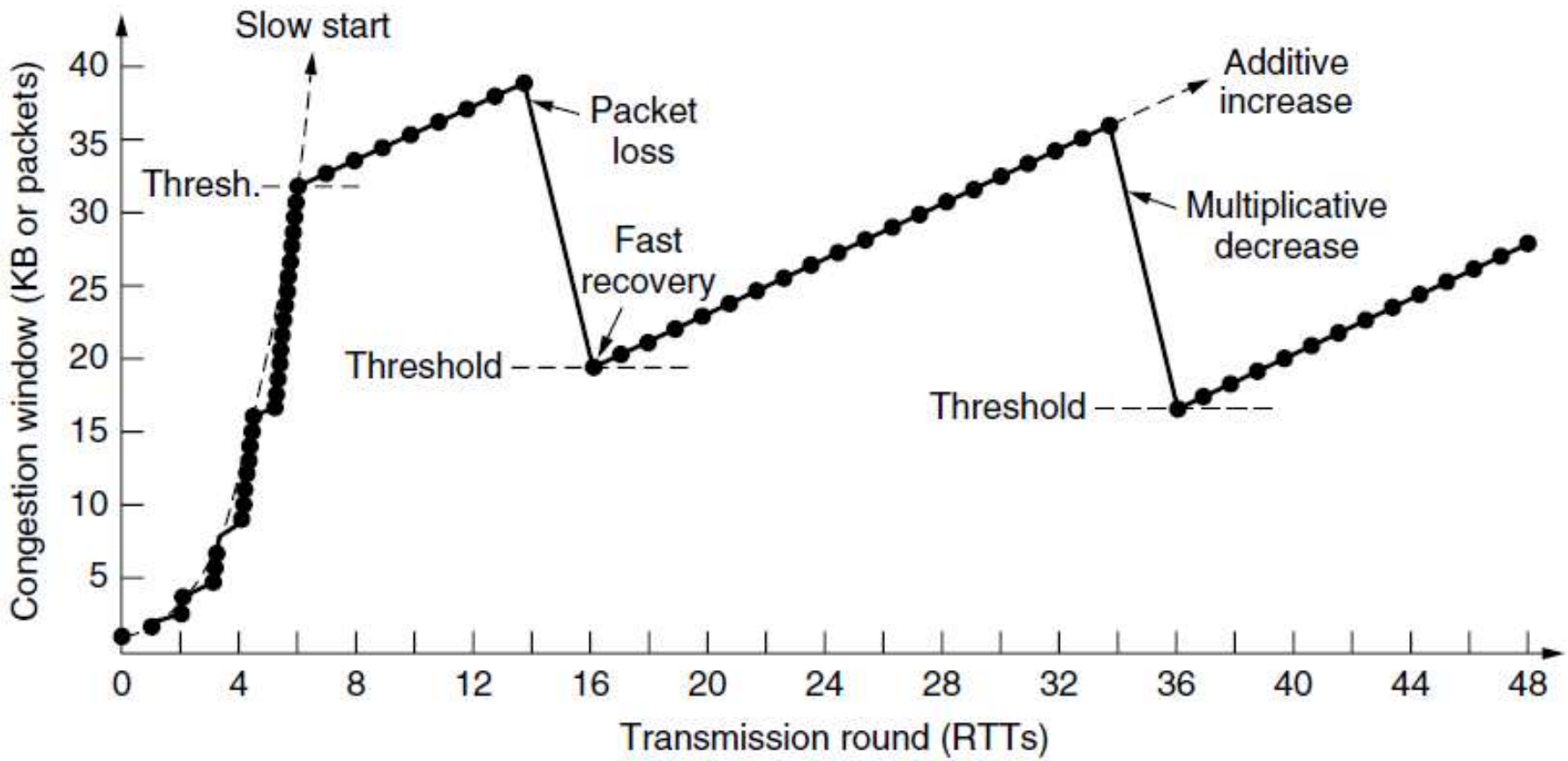
- **Drawback:** Timeouts are relatively long because they must be conservative. After a packet is lost, the receiver cannot acknowledge past it, so the acknowledgement number will stay fixed, and the sender will not be able to send any new packets into the network because its congestion window remains full.
- This condition can continue for a relatively long period until the timer fires and the lost packet is retransmitted. At that stage, **TCP slow starts again.**
- There is a quick way for the sender to recognize that one of its packets has been lost. As packets beyond the lost packet arrive at the receiver, they trigger acknowledgements that return to the sender.
- These acknowledgements bear the same acknowledgement number. They are called **duplicate acknowledgements**.

- TCP somewhat arbitrarily assumes that three duplicate acknowledgements imply that a packet has been lost. The identity of the lost packet can be inferred from the acknowledgement number as well.
- It is the very next packet in sequence. This packet can then be retransmitted right away, before the retransmission timeout fires.
- This heuristic is called **fast retransmission.**
- **Fast recovery** is a temporary mode that aims to maintain the ack clock running with a congestion window that is the new threshold, or half the value of the congestion window at the time of the **fast retransmission.**

sawtooth pattern of additive increase

- The upshot of this heuristic is that TCP avoids slow start, except when the connection is first started and when a timeout occurs.
- The latter can still happen when more than one packet is lost and fast retransmission does not recover adequately.
- Instead of repeated slow starts, the congestion window of a running connection follows a **sawtooth** pattern of additive increase (by one segment every RTT) and multiplicative decrease (by half in one RTT).
- This is exactly the AIMD rule that we sought to implement.

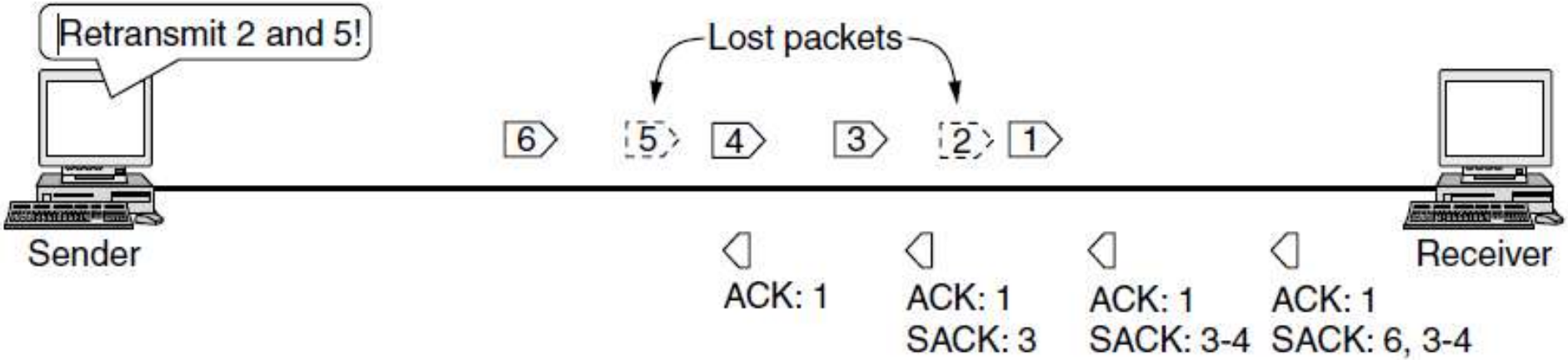
- The sawtooth behavior is shown in below figure.



Fast recovery and the sawtooth pattern of TCP Reno.

- The cumulative acknowledgement number does not provide the information of which packets have arrived and which packets have been lost.
- A simple fix is the use of **SACK (Selective ACKnowledgements)**, which lists up to three ranges of bytes that have been received. With this information, the sender can more directly decide what packets to retransmit and track the packets in flight to implement the congestion window.
- When the sender and receiver set up a connection, they each send the SACK permitted TCP option to signal that they understand selective acknowledgements.

Selective acknowledgements



The Future of TCP

- **The first issue with TCP is :** TCP does not provide the transport semantics that all applications want. For example, some applications want to send messages or records whose boundaries need to be preserved. Other applications work with a group of related conversations, such as a Web browser that transfers several objects from the same server.
- Still other applications want better control over the network paths that they use. TCP with its standard sockets interface does not meet these needs well.
- Essentially, the application has the burden of dealing with any problem not solved by TCP.
- This has led to proposals for new protocols that would provide a slightly different interface. Two examples are SCTP (Stream Control Transmission Protocol), defined in RFC 4960, and SST (Structured Stream Transport) (Ford, 2007).

- The second issue is congestion control.
- The signal might be round-trip time, which grows when the network becomes congested, as is used by FAST TCP (Wei et al., 2006). Other approaches are possible too, and time will tell which is the best.